CivicFix

Project Blueprint & Master Prompt for Claude Code

A detailed source of truth for the product workflow, roles and permissions, AI-agent boundaries, database entities, API modules, security rules, testing strategy, and backend-first implementation plan.

Primary implementation rule

Build and verify the backend in small, approved phases. Do not start frontend work until the backend workflows, permissions, tests, and API contracts are stable.

Prepared as a project reference document

Keep this PDF in the CivicFix repository and point Claude Code to it when starting a session.

How to use this document

This document is the product and implementation source of truth for CivicFix. Save it in the repository, for example as docs/CIVICFIX_PROJECT_BLUEPRINT.pdf. At the start of each Claude Code session, ask Claude to read it before proposing or changing implementation.

- Treat the finalized business rules in this document as requirements, not suggestions.
- Inspect the current repository and existing code before editing anything.
- Implement only the phase explicitly requested. Do not silently skip ahead.
- If an ambiguity blocks safe implementation, explain it and ask one focused question; otherwise make a conservative choice and document it.
- Do not claim a test, build, deployment, or integration passed unless it was actually run and the result is known.

1. Product vision and scope

CivicFix is an agentic AI civic-complaint platform. It turns citizens' reports about public problems into categorized, prioritized, assigned, trackable municipal work orders. It supports evidence-based resolution, human review for uncertain or disputed cases, notifications, feedback, and an audit trail.

Primary users and value

User group	What they need	CivicFix provides
Citizen	Report a problem and understand what happens next.	Complaint submission, status tracking, updates, feedback and dispute submission.
Department member	Understand department-wide issues and perform authorized work.	Department-scoped complaint visibility, personal/authorized work queue, progress updates and evidence submission.
Department manager	Allocate limited capacity and handle exceptions.	Workload view, assignments/reassignments, exception review, dispute decisions and metrics.
Platform administrator	Configure and govern the platform.	Department and account administration, policy configuration, audit and authorized cross-platform oversight.

Hackathon implementation scope

- Demonstrate one end-to-end flow reliably: complaint → AI analysis → validated assignment → work progress → evidence → verification or manager review → citizen update → audit history.
- Use simulated civic data and controlled test cases where real municipal integrations are unavailable.
- Do not integrate with live emergency dispatch or claim that an AI photo review proves physical safety or durable repair.
- Treat maps, advanced analytics, external municipal integrations and extensive notification channels as later enhancements unless explicitly approved.

2. Finalized business rules

Assignment model: hybrid

- The AI analyses the complaint and recommends category, priority, department, candidate duplicates and an assignment rationale.
- The backend—not the language model—checks deterministic rules and decides whether auto-assignment is permitted.
- Straightforward cases may be auto-assigned only when category and department are sufficiently clear, a suitable active member exists, workload/eligibility rules pass, and no review policy is triggered.
- Exceptions must be placed in a manager-review queue. Examples: uncertain category or department, no suitable available member, high urgency or possible hazard, multiple departments involved, ambiguous duplicate, or a policy requiring approval.

- The manager can approve, modify or reject the recommendation. Record the actor, decision, reason and time.
- Confidence thresholds, eligibility criteria and workload limits must be explicit configuration or documented deterministic rules—not invented by the AI at runtime.

Resolution model: AI-assisted, human accountable

- A department member submits completion notes and required evidence against the correct work order.
- The verification service checks evidence completeness and consistency and returns a structured assessment with reasons and uncertainty.
- If evidence is sufficient and configured policy allows automatic closure, the backend may close the work order/complaint and notify the citizen.
- If evidence is incomplete, ambiguous, contradictory, or potentially safety-critical, request more evidence or send it to the department manager.
- A citizen can dispute a resolution. Preserve the previous decision and evidence; reopen or move the case into the defined dispute workflow.
- If a manager rejects the resolution, return the work order to an appropriate active state, record the reason and notify relevant parties.
- AI must never claim that a photograph alone proves a physical repair is safe, complete or durable.

Department visibility model

- Department members may view complaints classified into their department, including unassigned complaints, according to the department visibility policy.
- Department managers can view and manage department-level queues and exceptions.
- Visibility is not permission to mutate data. Members may change only work orders assigned to them or actions explicitly authorized by policy; otherwise they can request reassignment or flag a blocker.
- Citizens can view their own complaints and relevant public-facing updates, not other citizens' private account data.
- Platform administrators have only the platform-wide permissions defined by policy; sensitive overrides must be audited.

3. Roles and authorization

Capability	Citizen	Department member	Department manager	Platform admin
Create a complaint	Yes	Only if policy permits	Only if policy permits	Only if policy permits
View complaints	Own complaints	Department scope	Department scope	Authorized platform scope
View work orders	Relevant public status only	Assigned/authorized work; department view as policy allows	All within managed department	Authorized oversight
Accept/start/update work	No	Only authorized work orders	Within delegated department scope	Exceptional override only, audited
Assign/reassign	No	Request only	Yes, within department	Configure/override according to policy; audited
Submit evidence	No	Authorized work orders	If explicitly permitted	If explicitly permitted
Dispute resolution	Yes, own complaint	No decision authority	Review and decide within department	Authorized oversight
Manage department memberships	No	No	Only if delegated	Yes
Read audit records	Relevant own history	Relevant authorized actions	Department scope	Authorized platform scope

Enforce authorization on the server for every request. Never trust a role, department ID, citizen ID, assigned-member ID, status or permission supplied by the client without checking it against authenticated identity and stored membership.

4. End-to-end workflow

4.1 Submit complaint

- Authenticated citizen submits title/description, location, optional category and optional media references.
- Validate required fields, accepted categories, coordinate/address shape, media size/type limits and request size.
- Create a unique complaint identifier and persist the initial REPORTED state.
- Return a confirmation with the complaint ID and safe public status.

4.2 Analyse complaint

- A controlled backend service sends the minimum required complaint data to the AI provider.
- AI returns structured fields: proposed category, priority, department, rationale, missing information, duplicate candidates, confidence/uncertainty and review flags.
- Validate the response against a schema and allowed enum values. Treat all model output as untrusted.
- Persist the AI analysis separately from the complaint and from final assignment decisions.

4.3 Duplicate handling

- Search likely duplicates using text/category and geographic proximity when location exists.
- AI may rank candidate matches, but backend rules or a human review must determine whether to link them.
- Do not delete or overwrite a citizen's original complaint. Preserve its history and connect it to the canonical/related issue with an explicit relationship and audit event.
- Ambiguous duplicate decisions go to review rather than automatic merging.

4.4 Decide and assign

- Backend checks department/category mapping, active membership, eligibility, workload, urgency, cross-department flags and policy thresholds.
- For a straightforward case, create or update the work order and assign an eligible member using deterministic selection rules.
- For an exception, place the case in a manager-review state/queue with the AI recommendation and reason visible.
- Use database transactions or equivalent safeguards where multiple writes must succeed together.
- Notify the citizen about meaningful status changes without exposing internal-only notes or sensitive staff data.

4.5 Work execution

- Assigned member accepts the task, starts work, adds notes, or marks it BLOCKED with a reason.
- Member can request reassignment but cannot silently assign the task to someone else.
- Every status change must validate the current state, actor permissions and allowed transition.
- Record timestamps and actor identity for acceptance, start, blockers and reassignment.

4.6 Submit resolution evidence

- Member submits completion notes and required evidence references for the specific work order.
- Evidence metadata records uploader, timestamp, type, storage key/reference and association with the work order.
- Store image/video binaries in object storage or a suitable file service; store metadata and references in MongoDB.

- Move the work order to RESOLUTION_SUBMITTED only when required fields and evidence requirements pass validation.

4.7 Verify resolution

- Verification checks required evidence, relevance to the original issue, completeness, consistency and any configured policy.
- Persist a structured verification result: outcome, confidence/uncertainty, reasons, missing evidence and whether human review is required.
- Sufficient evidence can trigger policy-approved automatic closure; uncertain cases request more evidence or go to manager review.
- Do not use the model's free-text response alone as the authorization to close a case.

4.8 Citizen update, feedback and dispute

- On approved resolution, update the complaint and work order consistently, notify the citizen and append audit events.
- Citizen may submit feedback or dispute the resolution with a reason and optional evidence.
- A dispute creates a trackable review record; the manager records a decision and rationale.
- Reopening must preserve the original resolution, prior evidence and all prior audit entries.

5. State machine and transition rules

Status	Meaning / use
REPORTED	Complaint has been submitted and persisted.
UNDER_REVIEW	AI analysis, duplicate review, or human triage is underway.
ASSIGNED	A work order has an approved/valid assignment.
ACCEPTED	Assigned member accepted responsibility.
IN_PROGRESS	Work has started.
BLOCKED	Work cannot proceed; a reason and next action are required.
RESOLUTION_SUBMITTED	Member submitted completion notes/evidence.
VERIFICATION_REQUIRED	More evidence or manager review is required.
RESOLVED	Resolution passed configured verification/approval.
CLOSED	Complaint lifecycle is complete under the configured policy.
REJECTED	Complaint was rejected with a reason and appropriate notification.

Not every complaint uses every status. Define explicit transitions, permitted actors and required fields for each transition. Reassignment, duplicate linking, dispute, reopening and rejection should be explicit actions, not arbitrary status edits. Keep complaint and work-order status separate where their lifecycles differ.

6. Data model / MongoDB entities

User

Identity and account state: name, normalized email, password hash or external-auth identifiers, account status, timestamps. Never store plaintext passwords. Public registration must not grant manager/admin privileges.

Department

Name, description, supported categories, coverage/operating configuration and active status. Define how category-to-department mapping is configured.

DepartmentMembership

User reference, department reference, membership role, active status, eligibility/skills if needed, and membership timestamps. Supports scoped authorization and potentially multiple memberships.

Complaint

Public ID, citizen reference, title/description, location, optional selected and AI-confirmed category, priority, status, media references, duplicate/related complaint links, timestamps and public-facing history references.

WorkOrder

Complaint reference, responsible department, assigned member, assignment source/reason, priority, due/target times if configured, status, work notes, blocker reason, completion metadata and timestamps.

AIAnalysis

Complaint reference, model/provider/version where available, structured category/priority/department recommendations, duplicate candidates, rationale, confidence/uncertainty, missing fields, review flags, timestamps and analysis version.

Evidence

Work-order and complaint references, uploader, evidence type, storage key/URL reference, file metadata, capture/upload timestamps, notes, verification state and retention metadata as appropriate.

AuditLog

Actor type and ID (human/system/AI), action, target type/ID, timestamp, correlation/request ID, reason, and safe before/after diff. Append-only through normal application workflows; restrict access.

Notification

Recipient, event type, related complaint/work order, safe message/payload, read state, created/delivered timestamps and delivery status.

Dispute

Complaint/work-order reference, citizen submitter, reason, optional evidence, status, assigned reviewer, manager decision/reason, timestamps and links to prior resolution. Preserve history.

Feedback (optional separate model)

Complaint reference, citizen, rating/comments, submitted time and moderation state if feedback should be distinct from formal disputes.

Data integrity guidance

- Add indexes for normalized user email, public complaint ID, common complaint filters (status/category/department/time), work-order queues, membership lookups, notifications and audit targets.
- Choose delete/archive behavior carefully. Prefer soft-deactivation or retention policies over cascading deletion of operational/audit history.
- Use schema validation, unique constraints where appropriate, consistent UTC timestamps and explicit enums.
- Avoid storing large files or secrets in ordinary documents. Sanitize sensitive fields from logs and API responses.
- Document how a complaint is associated with a department before assignment; recommended rule: after classification, department-scoped visibility applies to the department selected by the validated classification/mapping, while uncertain/cross-department cases enter manager review.

7. API modules and proposed endpoints

Module	Example routes	Responsibility / guardrails
Health	GET /api/v1/health	Report API status and database dependency status without leaking secrets.
Auth	POST /api/v1/auth/register; POST /api/v1/auth/login; POST /api/v1/auth/logout; GET /api/v1/auth/me	Authentication and current identity. Registration policy must restrict privileged roles.
Users	GET /api/v1/users/me; PATCH /api/v1/users/me	Safe profile operations; no self-service role escalation.
Departments	GET /api/v1/departments; GET /api/v1/departments/:id	Public/authorized department information; admin-only configuration routes.
Memberships	GET /api/v1/departments/:id/members; POST/PATCH membership routes	Managers/admins manage memberships only within authorized scope.
Complaints	POST /api/v1/complaints; GET /api/v1/complaints; GET /api/v1/complaints/:id	Create and query complaints with citizen ownership and department scoping.
AI analysis	POST /api/v1/complaints/:id/analyze; GET /api/v1/complaints/:id/analysis	Run/read structured AI analysis; protect against repeated expensive calls and validate output.
Assignments	POST /api/v1/work-orders/:id/assign; POST /api/v1/work-orders/:id/reassign; POST /api/v1/assignments/:id/decision	Backend-controlled assignment, manager approvals and explicit decision records.
Work orders	GET /api/v1/work-orders; GET /api/v1/work-orders/:id; POST action endpoints for accept/start/block/complete	Queue filtering and explicit lifecycle actions; do not expose arbitrary status patching.
Evidence	POST /api/v1/work-orders/:id/evidence; GET /api/v1/work-orders/:id/evidence	Secure upload initiation/reference and metadata; enforce ownership and file limits.
Verification	POST /api/v1/work-orders/:id/verify; POST /api/v1/work-orders/:id/request-evidence; POST /api/v1/work-orders/:id/manager-decision	AI assessment plus policy-based backend decision and manager review.
Disputes	POST /api/v1/complaints/:id/disputes; GET /api/v1/disputes; POST /api/v1/disputes/:id/decision	Citizen disputes, scoped review queue and recorded manager decisions.
Notifications	GET /api/v1/notifications; PATCH /api/v1/notifications/:id/read	Only recipients can read/mark their own notifications, except authorized administrative access.
Analytics	GET /api/v1/analytics/department; GET /api/v1/analytics/platform	Aggregate statistics with department/platform scope and appropriate privacy.
Audit logs	GET /api/v1/audit-logs; GET /api/v1/audit-logs/:id	Authorized, filtered access to audit events; no ordinary edit/delete endpoint.

These are proposed contracts, not existing implemented endpoints. Finalize request/response schemas and status codes during the relevant implementation phase. Prefer explicit action endpoints over a generic PATCH that lets clients set arbitrary statuses.

8. Agentic AI architecture and safe tool use

The AI agent is a planner/recommender operating through a controlled set of backend tools. It does not receive database credentials or unrestricted database access.

Suggested tools

- findSimilarComplaints(complaintId or validated search input)
- getDepartmentWorkload(departmentId)
- getEligibleMembers(departmentId, taskType)
- recommendPriority(validated complaint facts)
- recommendDepartment(validated complaint facts)

- requestManagerReview(complaintId, reason, structured recommendation)
- createWorkOrder(validated assignment decision)
- submitForVerification(workOrderId, evidence references)
- notifyStakeholders(event, authorized recipients)

AI tool contract

- Define input and output schemas for every tool; reject malformed or unexpected arguments.
- Check authenticated/system actor, department scope, entity ownership, current state and policy before executing a write.
- Separate recommendations from committed business decisions. Persist the recommendation, backend decision and reason independently.
- Make write actions idempotent where retries could create duplicate work orders, notifications or audit events.
- Use timeouts, bounded retries, rate limits and safe failure behavior. If AI is unavailable, preserve the complaint and route it to a fallback/manual review queue.
- Treat complaint descriptions, uploaded text and AI output as untrusted input. They must never override system rules or permissions.
- Log tool name, result category, actor type and correlation ID without logging secrets or unnecessary personal data.

9. Security, privacy and reliability requirements

- Validate every request at the API boundary and validate model output before it reaches the database.
- Apply authentication, role-based authorization, department scope, citizen ownership and assignment checks on the server.
- Hash passwords with an established password-hashing library; use secure session/cookie or token practices selected during the auth phase.
- Keep secrets in environment variables or a secret manager. Provide .env.example with placeholders only; never commit .env or provider keys.
- Use appropriate CORS allowlists, security headers, request-size limits, rate limits for sensitive routes, and upload type/size checks.
- Prevent mass assignment: map permitted request fields explicitly rather than spreading request bodies into database updates.
- Avoid leaking stack traces, database errors, tokens, passwords, private notes or internal AI prompts to clients.
- Audit sensitive actions, manager decisions, assignment changes, status changes and AI-triggered writes.
- Use transactions or carefully designed recovery/idempotency where several related records must change together.
- Provide meaningful health checks and graceful shutdown. Distinguish API liveness from database readiness.
- If AI, database or file storage is unavailable, fail safely, retain valid submitted reports where possible and queue/manual-review rather than fabricate success.

10. Recommended backend folder structure

```
backend/
src/
config/ # env, database, provider configuration
models/ # Mongoose schemas and indexes
routes/ # versioned route registration
controllers/ # HTTP request/response handling only
services/ # business rules and use cases
agents/ # orchestration for AI workflows
tools/ # narrow, validated AI-callable backend tools
middleware/ # auth, authorization, errors, request context
validators/ # request and AI-output schemas
policies/ # status transitions, assignment/closure rules
utils/ # shared safe utilities
app.js # Express app composition
server.js # process startup and graceful shutdown
tests/
.env.example
README.md
package.json
```

Keep controllers thin. Put business rules in services/policies so they can be tested without HTTP. Keep AI orchestration separate from deterministic policy enforcement. Adjust this structure to the existing repository rather than duplicating or replacing working architecture without a reason.

11. Phased implementation plan — backend first

Phase 1 — Foundation

Inspect repo, Node version and package manager; establish Express app, environment validation, MongoDB connection, structured logging, centralized error handling, not-found handling, validation infrastructure, security headers, CORS configuration, request limits, health endpoint and tests. No business models, auth, AI or frontend unless already present and explicitly required.

Phase 2 — Identity and access

Implement User, Department and DepartmentMembership models; authentication; current-user endpoint; role/department authorization middleware; seed/development setup if needed; tests for role escalation, ownership and cross-department access.

Phase 3 — Complaints and work orders

Implement Complaint and WorkOrder models; complaint submission/list/detail; department-scoped visibility; work queues; explicit status-transition services; assignment and reassignment requests; audit hooks for critical actions.

Phase 4 — AI analysis and hybrid assignment

Implement AI provider abstraction, structured classification/priority/duplicate recommendations, analysis persistence, deterministic assignment policy, manager-review queue, narrow tool execution and AI-unavailable fallback.

Phase 5 — Evidence, verification and disputes

Implement Evidence and Dispute (plus optional Feedback), secure file-reference workflow, verification assessment, policy-based closure, manager decisions, citizen disputes/reopening and relevant notifications/audit events.

Phase 6 — Quality and deployment readiness

Run unit/integration/API tests, permissions matrix tests, invalid-transition tests, AI malformed-output tests, provider failure tests, duplicate/idempotency tests; document API schemas/OpenAPI; verify environment setup, production build/start, logs and deployment assumptions.

Phase 7 — Frontend (only after approval)

Build citizen interface first against stable documented APIs; then department portal; then manager/admin views. Do not begin this phase until backend acceptance criteria are met and user approves.

12. Definition of done

- Each implemented phase has a clear scope, tests and a concise summary of changed files.
- No protected endpoint relies on frontend hiding buttons for security.
- All important status transitions and assignment/verification decisions have permission checks and audit events.
- AI outputs are structured, validated, persisted and separated from final backend decisions.
- Core happy paths and critical negative paths have automated tests.
- API contracts and environment setup are documented; secrets are not committed.
- A clean checkout can install dependencies, configure environment, run tests and start the backend following the README.
- No claim of completion is made for a feature that has only been stubbed, mocked or documented.

13. Master prompt for Claude Code

Copy the prompt below into Claude Code whenever you want it to use this document as the implementation specification. At the end, replace [PHASE TO IMPLEMENT] with the one phase you want it to work on.

```
You are the senior backend engineer for CivicFix, an agentic AI civic-complaint and municipal work-order platform.

SOURCE OF TRUTH
1. First read the entire file `docs/CIVICFIX_PROJECT_BLUEPRINT.pdf` (or the actual path where this PDF is stored).
2. Treat its finalized business rules, role model, state machine, AI boundaries, security requirements and phased plan as requirements.
3. Inspect the current repository before editing. Preserve working code and conventions unless a change is justified.
4. If the PDF cannot be read by your environment, say so and ask me to provide a text/Markdown copy; do not pretend you read it.

PRODUCT RULES
- Assignment is hybrid: AI recommends; backend policy auto-assigns straightforward cases; exceptions go to a department manager.
- Resolution is AI-assisted: AI checks evidence; uncertain cases and citizen disputes go to a manager.
- Department members can see complaints in their department after validated classification, including unassigned cases according to policy. Visibility does not grant mutation rights.
- Citizens see only their own complaints and appropriate public-facing updates.
- Backend authorization and deterministic policy—not the frontend or LLM—control permissions, assignments, status transitions and closure.
- AI has no unrestricted database access. It calls narrow, schema-validated backend tools.
- Preserve original complaints, prior decisions and audit history. Never silently erase a complaint or dispute.
- Never claim photo-only AI verification proves a physical repair is safe or durable.

ENGINEERING RULES
- Backend first. Do not implement frontend work unless I explicitly approve the frontend phase.
- Work only on [PHASE TO IMPLEMENT]. Do not silently advance to later phases.
- Begin by summarizing the current repository structure, relevant existing code, and a small implementation plan.
- Use the repository's actual package manager, runtime and code conventions. Do not replace tooling without explaining why.
- Use modular Express/Mongoose architecture if consistent with the existing repository. Keep HTTP controllers thin; place business logic in services/policies.
- Validate request input and AI output; use explicit allowlists for mutable fields; prevent mass assignment.
- Apply authentication, role, department scope, citizen ownership and work-order assignment checks on every relevant endpoint.
- Add tests for success cases, permission denials, invalid state transitions, malformed AI outputs and relevant failure paths.
- Keep secrets out of source control and logs. Update `.env.example` with placeholders only.
- Do not fabricate integrations, API keys, test results, deployment status or implemented features.
- If an implementation decision is ambiguous but non-blocking, choose a conservative approach and document it. If it blocks safe implementation, ask one focused question before making irreversible changes.
```

WORK PROCESS
1. Inspect repository and report existing state before editing.
2. List intended files and behavior changes for this phase.
3. Implement only the agreed scope.
4. Run relevant tests, lint/type checks and build commands that actually exist in the repository.
5. Report commands run and exact outcomes; distinguish passed, failed, skipped and not available.
6. Summarize changed files, endpoints/models added, security considerations, known limitations and next recommended phase.
7. Stop and wait for my approval before proceeding to another phase.

PHASE TO IMPLEMENT NOW: [PHASE TO IMPLEMENT]

Acceptance criteria for this phase:
- Requirements in the blueprint for this phase are met.
- Tests cover the key behavior and relevant denial/failure paths.
- Documentation is updated.
- No out-of-scope frontend or future-phase implementation is added.

14. Recommended first prompt for the current stage

If the repository is not yet built out, start with Phase 1 only. Use this focused instruction after saving the PDF:

```
Read `docs/CIVICFIX_PROJECT_BLUEPRINT.pdf` completely, inspect the repository, and implement Phase 1 — Foundation only. First report the repo state and plan. Build a maintainable Express backend foundation with environment validation, MongoDB/Mongoose connection handling, accurate health/readiness reporting, centralized errors, not-found handling, request validation infrastructure, structured logging, security headers, environment-configured CORS, request-size limits, scripts and automated tests. Follow the existing package manager and conventions. Do not implement authentication, business models, complaint/work-order workflows, AI, or frontend in this phase. Do not stop the server from starting in development solely because MongoDB is temporarily unavailable if the design can safely report degraded readiness, but never report the database as healthy when it is disconnected. Run the tests and relevant checks, report exact results and changed files, then stop for approval.
```

15. Suggested repository placement

```
CivicFix/
docs/
CIVICFIX_PROJECT_BLUEPRINT.pdf
backend/
frontend/ # create/use later, after backend approval
README.md
.gitignore
```

If Claude Code cannot reliably extract text from a PDF, keep a Markdown copy of the same blueprint (for example docs/CIVICFIX_PROJECT_BLUEPRINT.md) as the machine-readable companion. The PDF is convenient for reading; Markdown is often easier for coding tools to search and reference.

Appendix — Decisions that remain configurable

- Exact AI confidence thresholds and priority scoring policy.
- Workload formula, member eligibility/skills and tie-breaking rules for automatic assignment.
- Whether a specific category requires mandatory manager approval.
- Evidence requirements by complaint category and which categories permit automatic closure.
- Complaint and work-order status transition matrix, including duplicate-link, rejection, reopening and dispute transitions.
- Authentication/session strategy and deployment environment details.
- File/object-storage provider, retention policy and permitted media types.
- Department coverage/location rules and cross-department ownership handling.

These values should be explicitly configured or agreed during the relevant phase. Do not let the LLM invent them dynamically.